

Anexo A

Santiago Rosas Mogollón

Marzo 2025

1. Control y Configuración del Motor y Driver

Para el control tanto del motor como del Driver se realizó una revisión a fondo de los manuales puestos a disposición del público por Rtelligent. En ellos se especifican tanto el modo de operación de las palancas del Driver como las diferentes opciones de micro pasos que se pueden obtener con las combinaciones de Estados encendido o apagado de los mismos además de este se especifican las conexiones del motor del Driver qué fuente requieran el tipo de señal que aceptan especificaciones técnicas y una guía sobre las señales de salida del Driver.[1][2]

1.1. Configuración de las Palancas de Selección (Switches)

Sobre las palancas de la **sw5** a la **sw8** modifican el modo de operación del Driver:

- La palanca **sw5** modifica la dirección del motor manualmente.
- La palanca **sw6** apaga o enciende la función de filtro de pulso.
- La palanca **sw7** selecciona el modo de pulso que se va a tomar si es un modo de pulso simple o un modo de pulso doble. Para el montaje se utilizó el modo de pulso simple.
- La palanca **sw8** controla si el motor está en modo lazo cerrado o abierto, el lazo abierto puede ser utilizado para una prueba inicial del motor.

Así mismo, las palancas de la **sw1** hasta la **sw4** permiten controlar los pulsos por Revolución, que no son más que los micro pasos en los que se pueden dividir los pasos nominales del motor. Para el motor **57A3ED** que se utilizó, los pasos nominales eran 200, es decir, se necesitan 200 pulsos para dar un giro completo. Con respecto a la configuración utilizada de pulsos/rev, se seleccionó la mayor posible de 40,000 pulsos/rev, según como se indica tanto en el manual del Driver T60 como en la placa del mismo donde se encuentran todas las configuraciones posibles del Driver.

1.2. Conexiones del Sistema

En cuanto a las conexiones, basta con entender las conexiones del Driver para conseguir realizar el correcto ensamble del sistema. El Driver se divide en cuatro secciones de conexión:

1. **VDC / Alimentación:** Corresponde a la conexión hacia la fuente de alimentación. En este montaje se usó una fuente Drok de voltaje variable a 24 voltios.
2. **Motor:** Es la conexión directa con el stepper servomotor **57A3ED**. Esta puede ser encontrada en la calcomanía de información del motor. En este caso la conexión es:
 - **A+:** Rojo
 - **A-:** Azul

- **B+:** Verde
 - **B-:** Negro
3. **Encoder:** Tiene una conexión estandarizada para el Driver, la cual está distribuida de la siguiente manera:
- **EB+:** Verde
 - **EB-:** Amarillo
 - **EA+:** Marrón
 - **EA-:** Blanco
 - **Vcc:** Rojo
 - **GND:** Azul
4. **Signal:** Son las conexiones para el control del pulso suministrado, el cual controla el avance del motor y su dirección. La conexión realizada fue:
- PUL+ , DIR+ y ENA- al pin 9 PWM del Arduino Uno.
 - PUL- , DIR- y ENA+ al pin GND digital del mismo Arduino Uno.

Este Arduino fue programado con el código presentado en el **Apéndice A**.

1.3. Parámetros de Velocidad y Desplazamiento

Finalmente, la velocidad del motor se estableció en la más baja posible, la cual corresponde a 0,005 revoluciones por segundo. Dado el paso del actuador lineal de 5 mm por revolución, se obtiene un desplazamiento relativo entre mordazas de 0,025 mm cada segundo. Esto se logró generando un pulso cuadrado de 250 Hz y un 50 % de ciclo de trabajo.

Apéndice A: Código Arduino

El siguiente código fue implementado en la plataforma Arduino para el control de la frecuencia de los pulsos enviados al Driver:

```
1 // Definir el pin donde se generará el pulso
2 const int pinPulso = 9; // Pin 9 es válido en Arduino Uno
3 // Variable para almacenar la frecuencia deseada (en Hz)
4 int frecuencia = 0; // Frecuencia inicial
5
6
7 // Variable para almacenar el periodo del pulso (en microsegundos)
8 unsigned long periodo;
9
10 void setup() {
11     // Configurar el pin como salida
12     pinMode(pinPulso, OUTPUT);
13     // Iniciar la comunicación serial
14     Serial.begin(115200);
15     // Calcular el periodo inicial
16     periodo = 1000000 / frecuencia; // 1 segundo = 1,000,000
17     // microsegundos
18 }
19
20 void loop() {
21     if (Serial.available() > 0) {
22         String input = Serial.readStringUntil('\n');
23
24         if (input == "STOP") {
25             // Detener completamente los pulsos
26             digitalWrite(pinPulso, LOW);
27         } else {
28             // Procesar como frecuencia
29             frecuencia = input.toInt();
30             if (frecuencia > 0) {
31                 periodo = 1000000 / frecuencia;
32             }
33         }
34     }
35
36     if (frecuencia > 0) { // Solo generar pulsos si frecuencia > 0
37         digitalWrite(pinPulso, HIGH);
38         delayMicroseconds(periodo / 2);
39         digitalWrite(pinPulso, LOW);
40         delayMicroseconds(periodo / 2);
41     } else {
42         digitalWrite(pinPulso, LOW);
43     }
44 }
```

Referencias

- [1] Rtelligent. 2 phase closed loop stepper drive factory - china 2 phase closed loop stepper drive manufacturers, suppliers. <https://www.rtelligentglobal.com/2-phase-closed-loop-stepper-drive-oplen-loop-stepper-system/>. Accedido: 11 de marzo de 2025.
- [2] Rtelligent. Wholesale phase closed loop stepper motor 57 series factory and suppliers. <https://www.rtelligentglobal.com/phase-closed-loop-stepper-motor-57-series-product/>. Accedido: 11 de marzo de 2025.